

RVBL-2

A Multicycle RISC-V Processor

RV32I + Zmmul + Xicrc • 47 instructions • SkyWater Sky130A

ChampionCHIP eXperience: Phase 2 Design Submission Report

Equipe 13

Implemented on the ChipInventor platform • Taped out through OpenLane to signed-off GDSII

ISA	RV32I (37) + Zmmul (4) + Xicrc (3) = 47/47, 100 %	Die	0.404 mm² (630.15 × 640.87 μm)
Microarchitecture	6-state multicycle FSM, 3–5 cycles/instruction	Density	52.28 % final core utilisation
Validation firmware	PASS: x4 = 0x00000000 , 9/9 stages, 991 cycles	Cells / flops	17,708 / 1,393
Verification	7,384 checks (15 TBs) + 15,281 checks (platform suite)	Timing	setup +2.25 ns slow corner; hold +0.11 ns
Equivalence	410,400 lockstep comparisons, 0 mismatches	Signoff	0 DRC • 0 routing viol. • LVS match uniquely
Technology	sky130A, sky130_fd_sc_hd, no macros	Clock	33 ns constraint; closes at ≈32.5 MHz

Project demonstration video (Submission Guide §7 / §9)

<https://youtu.be/TODO-REPLACE-WITH-YOUR-VIDEO-ID>

Source, testbenches and full-resolution figures

<https://github.com/Niccc8/rvbl2-riscv-cpu-core>

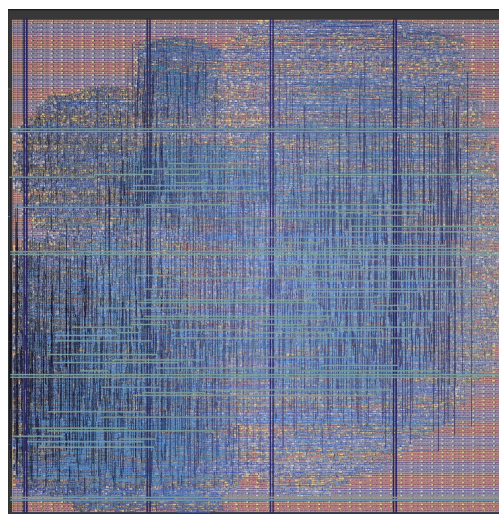


Figure 1: The finished RVBL-2 chip. The die is 630.15 × 640.87 μm and holds 17,708 standard cells in 0.404 mm². The pink bands at the top and bottom are power and filler rows. The blue area is the routed logic.

Every number in this report is measured, not estimated. Each one comes from a tool report or a test run in the submission, and this report says which. The section numbers follow the Submission Guide’s own topics 1 to 6.

Requirements compliance: where each requirement is answered

Guide §	Requirement	How it is answered	Page
1	Project Overview	Goal, multicycle architecture, 20 modules, general operation, RISC-V functionality implemented (§1)	3
2	Supported Instructions	Coverage grouped by ISA category, expected-vs-implemented summary, per-category lists, full 47-row encoding table (§2)	5
3	Datapath	Complete datapath with control unit, data flow between all modules, FSM, per-class cycle counts (§3)	6
4	Modules and Testbenches	All 17 ChipInventor blocks, their ports, their testbenches, and the ChipInventor block diagrams (§4)	8
5	OpenLane Results	Total area, density, signoff results and the final GDSII image (§5)	13
6	Firmware Demo	All 9 validation stages, waveforms, and verbatim testbench log output showing the run reached the end without errors (§6)	16
7	Video Demo	URL on the title page and in §7	2
8	Report Format	PDF, within 20 pages and 10 MB, covering topics 1–6	n/a
9	Expected Files	Deliverables map, §7	19

The central claim: the chip that was taped out is the design that was verified

Most submissions can show that their RTL passes a testbench and, separately, that *something* went through OpenLane. This project can show that they are the same design, by mechanical check rather than assertion. Figure 2 is the whole argument; every count in it was produced by a script in the repository and reproduced while this report was being written.

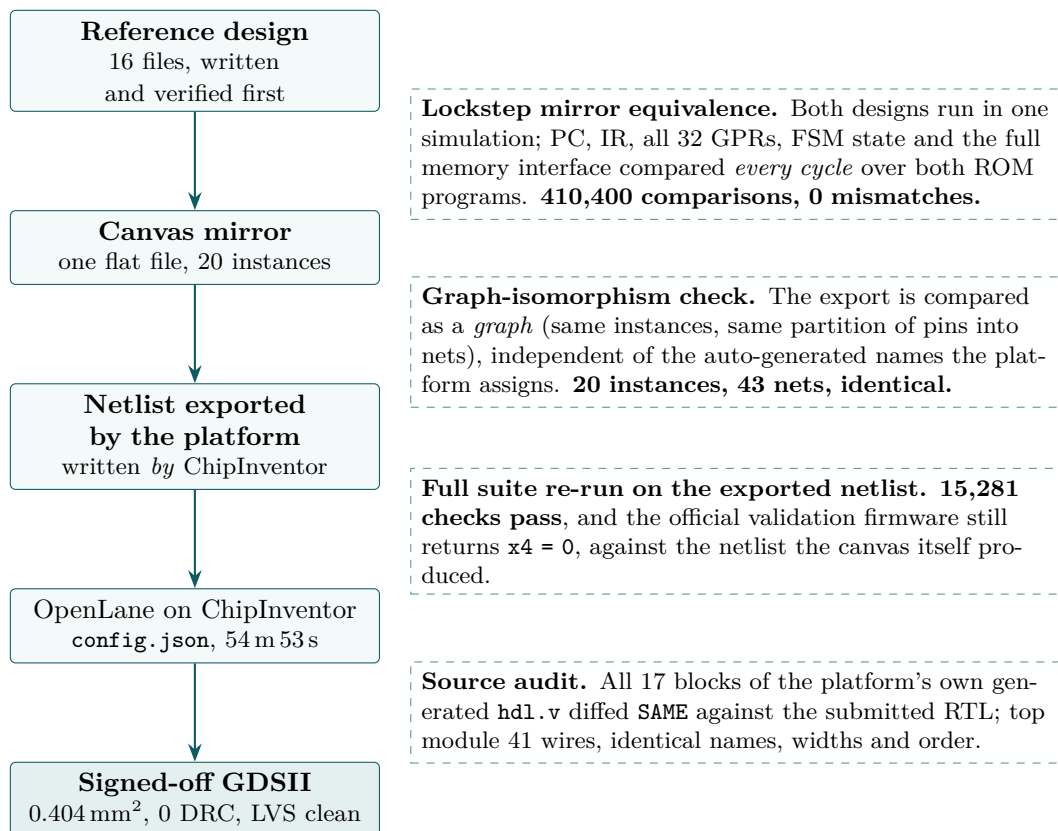


Figure 2: Unbroken evidence chain from reference RTL to signed-off GDSII. Each arrow is a mechanical check, not a claim: a swapped wire fails at the graph check rather than surfacing later as a puzzling simulation result. Every check in the chain is automated and is re-run before each submission.

1 Project Overview

In one paragraph. RVBL-2 is a 32-bit RISC-V processor core. We built it to the ChipInventor RVBL-2 Block Guide. It executes the complete RV32I base instruction set, the standard Zmmul multiply extension, and a custom CRC-16 extension called Xicrc. That is 47 instructions.

The core is *multicycle*. Each instruction takes three to five clock cycles, and the hardware is used again in each cycle. One arithmetic-logic unit (ALU), one memory port and one register write port therefore serve all instructions. Nothing is duplicated. The Block Guide requires this arrangement (§2.1) because it saves area. It also makes the design easy to verify: instructions never overlap, so no hazard can occur. There is no forwarding logic, no stall logic and no flush logic to get wrong.

We entered the core on the ChipInventor canvas one block at a time. We then exported it as a netlist and ran it through ChipInventor’s own OpenLane flow. The result is a clean, signed-off Sky130A layout.

1.1 What was implemented in the RISC-V core

- **The complete RV32I base instruction set**, all 37 instructions. This covers arithmetic and logic on registers and on immediate values, all five load widths, all three store widths, all six branch conditions, both jumps, both upper-immediate forms, and FENCE, ECALL and EBREAK. Loads and stores place each byte in the correct lane and extend the sign correctly.
- **Zmmul extension.** MUL, MULH, MULHSU, MULHU in a dedicated combinational 64-bit multiply array with per-operand signedness selection (Block Guide Table 10). No divider, as Zmmul specifies.
- **Xicrc custom extension.** CRCB, CRCH, CRCW, computing CRC-16/CCITT-FALSE (polynomial 0x1021, seed 0xFFFF, MSB-first, no reflection, no final XOR) over 8-, 16- and 32-bit input widths with *zero* latency, as Block Guide §3.1.3 requires.
- **Memory-mapped I/O.** An address decoder mediating between the core and two memories exactly as Block Guide §4.4 and Figure 3 describe: a combinational-read instruction ROM and a synchronous, byte-writable data SRAM.
- **Deterministic behaviour on every encoding.** An unrecognised {opcode, funct3, funct7} tuple decodes as a safe no-operation rather than locking the FSM, and rst_i aborts an in-flight memory write in the *same* cycle it is asserted.
- **Exactly two external pins**, clk_i and rst_i, as Block Guide Table 5 mandates. §4.2.1 explains how a design with no data outputs is kept observable and survives synthesis.

1.2 Main modules and general operation

The design is 17 block definitions placed as 20 instances (the 32-bit 2-to-1 multiplexer is used four times). Table 4 in §4 gives every block’s ports and its testbench; the datapath they form is Figure 5.

The core repeats one loop. Each step of the loop is one clock cycle.

1. **FETCH.** The program counter gives an address to the ROM through the decoder. The ROM returns an instruction word. The instruction register (IR) stores that word.
2. **DECODE.** The IR now holds its value until the next FETCH. It drives the register address ports, the immediate generator, and the decode logic in the control unit.
3. **EXECUTE.** The ALU calculates one of three things: an arithmetic result, a memory address, or a branch target. At the same time the multiplier, the CRC unit and the branch comparator calculate their own answers from the same two operands.
4. **MEMORY.** Loads and stores use this cycle to reach the data memory, which is synchronous. Other instructions skip it.
5. **WRITEBACK.** Any instruction that produces a register result uses this cycle to write it.

The core writes the program counter in the last cycle of each instruction. This is why branch and jump targets stay relative to the *original* address of the instruction, and not to PC+4.

Why there is no ALUOut or memory-data register

A textbook multicycle design adds two registers. One holds the ALU result. The other holds the word read from memory. This design has neither, and does not need them.

The reason is simple. The instruction register does not change until the next FETCH. Every signal that comes from it is therefore steady for the whole instruction. This includes `rs1_data`, `rs2_data`, the immediate value, and the ALU result. A register that holds a value which is already steady adds nothing.

Removing the two registers saves 64 flip-flops and does not change the behaviour. One rule must hold for this to be safe: no cycle may change the ALU operand multiplexers in the middle of an instruction. This is why PC+4 has its own small adder. It does not borrow an ALU cycle.

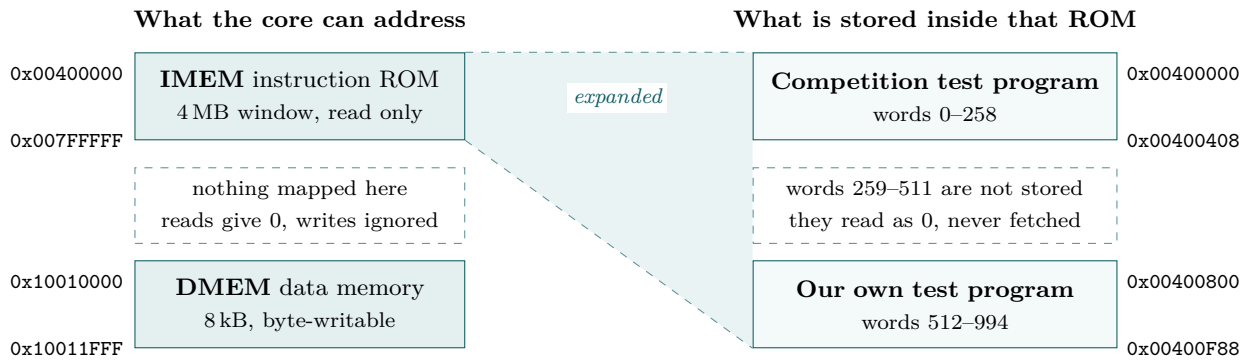


Figure 3: The memory map, and the two programs held in the instruction ROM. The core reads instructions from one 4 MB window and reads and writes data in a separate 8 kB window. Addresses outside both windows are safe: they read as zero and ignore writes. The shaded wedge shows that the right-hand column is an expanded view of the ROM only. The ROM stores 742 words, which is 259 words of the competition test program plus 483 words of our own test program. The 253 words between the two programs are not stored at all and always read as zero. The competition program must start at the reset address, because it calculates its own data-memory base from the program counter and therefore cannot be moved. Our own program does not do this, so it sits above.

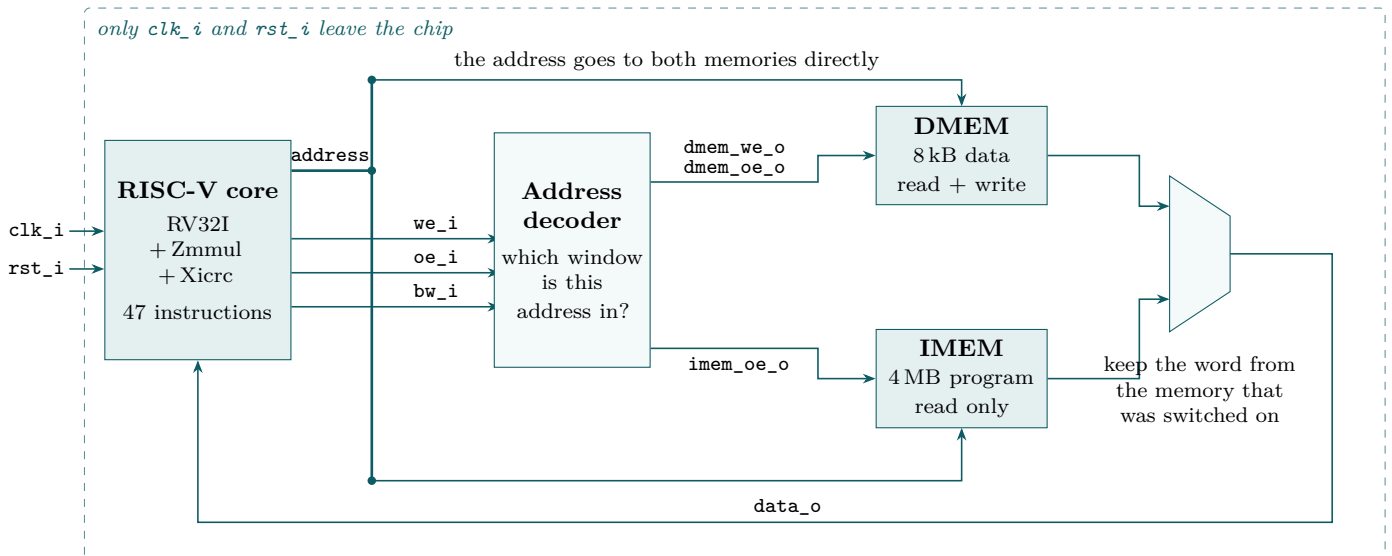


Figure 4: How the parts are connected. The core makes one memory request at a time. It sends an address, the data to be written, a read strobe (`oe_i`), a write strobe (`we_i`) and four byte-enables (`bw_i`). The address decoder compares the address against the two windows of Figure 3 and switches on no more than one memory. It switches DMEM on for a read or a write, but IMEM only for a read, because the program memory has no write port. The decoder then keeps the word from the memory it switched on and sends it back to the core as `data_o`. If the address is in neither window, the decoder switches on neither memory and returns zero, so a wrong address cannot damage stored data. Two details differ from a textbook drawing: the address reaches the memories directly instead of passing through the decoder, and the multiplexer drawn on the right is part of the address decoder block, not a block of its own.

2 Supported Instructions

In one paragraph. The Submission Guide names ten instruction categories. Together they contain 47 instructions. All 47 are implemented. Each one has a directed test that passes. Table 1 is the coverage summary in the form the Guide asks for. It also lists the instructions in each category. Table 2 gives the bit pattern of every instruction.

Table 1: ISA coverage summary with the instructions implemented in each category. The expected counts come from the Submission Guide (§2). The implemented counts are what the control unit block decodes. A passing test covers each one.

Category	Exp.	Impl.	Cov.	Instructions implemented
Arith./Logic (Register)	10	10	100 %	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
Arith./Logic (Immediate)	9	9	100 %	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
Load	5	5	100 %	LB, LH, LW, LBU, LHU
Store	3	3	100 %	SB, SH, SW
Branch	6	6	100 %	BEQ, BNE, BLT, BGE, BLTU, BGEU
Jump	2	2	100 %	JAL, JALR
Upper Immediate	2	2	100 %	LUI, AUIPC
System / Synchronization	3	3	100 %	ECALL, EBREAK, FENCE
Multiplication (Zmmul)	4	4	100 %	MUL, MULH, MULHSU, MULHU
CRC (Xicrc)	3	3	100 %	CRCB, CRCH, CRCW
Total	47	47	100 %	

2.1 Encoding of every implemented instruction

R-type values come from the Block Guide (Tables 7–9); I/S/B/U/J values are the public RV32I base encodings. Every row was cross-checked against the decode `case` statements in `control_unit.v`. This table is what the hardware matches on, not a separate statement of intent. Note that all seven R-type families share opcode 0110011 and are disambiguated by `funct7` alone (0000000/0100000 base ALU, 0000001 Zmmul, 1000000 Xicrc), which is why the decoder is a two-level structure rather than a 47-way lookup.

2.2 How coverage was established

The claim is not that the decoder *contains* 47 cases; it is that all 47 execute correctly. Three independent mechanisms establish that:

- **58 signature tests** in `tb_firmware` compute actual XOR expected into a dedicated data-memory word per test, with every expected value computed independently in Python rather than by the RTL. There are 58 rather than 47 because loads and stores are tested at *every* byte lane (`address[1:0] = 0/1/2/3`) and every branch condition is tested in both the taken and the not-taken direction. JAL, JALR, AUIPC, FENCE, ECALL and EBREAK, which produce no comparable register result, are checked structurally.
- **A complete decode sweep** drives 704 different instruction encodings through the real state machine. Invalid encodings are included on purpose. Each one is checked twice: which state follows EXECUTE, and whether the register write is enabled. The expected answers come from a separate model written from the instruction tables. That model was not copied from the design.
- **The organisers' own validation firmware**, which exercises RV32I, all four multiplies and all three CRC widths and returns `x4 = 0` (§6).

Table 2: Full encoding table for all 47 implemented instructions (Submission Guide §2). Every row was read back from the decode logic in the control unit block, so the table shows what the hardware matches on, not what was intended.

#	Instr.	Type	Opcode	f3	funct7	#	Instr.	Type	Opcode	f3	funct7
1	ADD	R	0110011	000	0000000	25	SB	S	0100011	000	n/a
2	SUB	R	0110011	000	0100000	26	SH	S	0100011	001	n/a
3	SLL	R	0110011	001	0000000	27	SW	S	0100011	010	n/a
4	SLT	R	0110011	010	0000000	28	BEQ	B	1100011	000	n/a
5	SLTU	R	0110011	011	0000000	29	BNE	B	1100011	001	n/a
6	XOR	R	0110011	100	0000000	30	BLT	B	1100011	100	n/a
7	SRL	R	0110011	101	0000000	31	BGE	B	1100011	101	n/a
8	SRA	R	0110011	101	0100000	32	BLTU	B	1100011	110	n/a
9	OR	R	0110011	110	0000000	33	BGEU	B	1100011	111	n/a
10	AND	R	0110011	111	0000000	34	JAL	J	1101111	n/a	n/a
11	ADDI	I	0010011	000	n/a	35	JALR	I	1100111	000	n/a
12	SLTI	I	0010011	010	n/a	36	LUI	U	0110111	n/a	n/a
13	SLTIU	I	0010011	011	n/a	37	AUIPC	U	0010111	n/a	n/a
14	XORI	I	0010011	100	n/a	38	ECALL	I	1110011	000	imm=0x000
15	ORI	I	0010011	110	n/a	39	EBREAK	I	1110011	000	imm=0x001
16	ANDI	I	0010011	111	n/a	40	FENCE	I	0001111	000	NOP
17	SLLI	I	0010011	001	0000000 [†]	41	MUL	R	0110011	000	0000001
18	SRLI	I	0010011	101	0000000 [†]	42	MULH	R	0110011	001	0000001
19	SRAI	I	0010011	101	0100000 [†]	43	MULHSU	R	0110011	010	0000001
20	LB	I	0000011	000	n/a	44	MULHU	R	0110011	011	0000001
21	LH	I	0000011	001	n/a	45	CRCB	R	0110011	000	1000000
22	LW	I	0000011	010	n/a	46	CRCH	R	0110011	001	1000000
23	LBU	I	0000011	100	n/a	47	CRCW	R	0110011	010	1000000
24	LHU	I	0000011	101	n/a						

[†] occupies imm[11:5], not a true funct7 field; the decoder requires exactly these values, so SLLI/SRLI/SRAI with any other value in that field fall through to the illegal/no-operation default.

3 Datapath and Control Unit

In one paragraph. Figure 5 shows the complete datapath and the control unit. Solid lines carry data. Dashed lines are control signals from the finite-state machine (FSM). Every box is a real block on the ChipInventor canvas, and every line is a real wire. This is the design itself, not a simplified drawing of it. Read one point first. The core has only *one* memory port. The FSM shares that port over time between instruction fetch and data access, and the address decoder sends each request to the correct memory. This sharing is the reason the Block Guide requires a multicycle design.

3.1 Control unit: the finite-state machine

Block Guide §3.2 names four phases. This design uses six states: the four named phases, plus RESET, plus a MEMORY state. The MEMORY state is not decoration. Block Guide §4.3 specifies the data memory as synchronous (“the operation will only be completed after a clock cycle”), so a load physically cannot present an address and consume the result in the same cycle. Six states encode in three bits, so the addition costs nothing.

3.2 Control signals and datapath multiplexers

The control unit produces every select signal in the datapath. Table 3 lists each multiplexer, the values it can choose between, and the signal that makes the choice. Four of the five multiplexers choose between two values. The writeback multiplexer chooses between five, because five different units can produce a register result. One signal, `op_size`, is not a select at all: it is `funct3` republished by the control unit so that four blocks read the width field from one source instead of four.

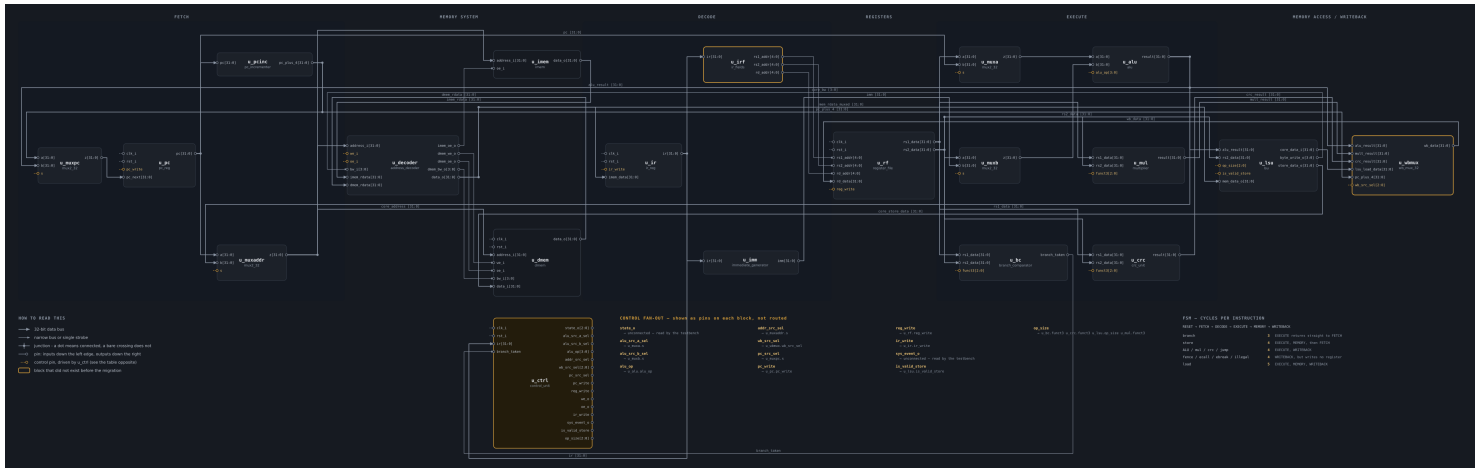
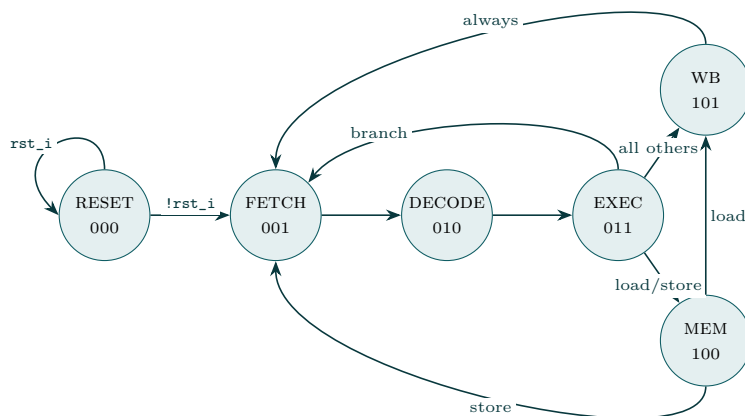


Figure 5: The complete RVBL-2 datapath and control unit, exported from the design itself. The columns follow the order of a single instruction, left to right: FETCH, the memory system, DECODE, the registers, EXECUTE, and finally memory access and WRITEBACK. Every box is one of the 20 block instances on the canvas and every line is one of the 43 nets between them, made from 72 connections. The control unit sits at the bottom. Its outputs are printed beside it as a list rather than drawn as 15 separate lines, because each one reaches almost every block above. Three panels at the bottom left give the drawing key, the control fan-out, and the number of cycles each class of instruction takes. One ALU serves arithmetic, address calculation and branch targets. One memory port serves both instruction fetch and data access. The multiplier, the CRC unit and the branch comparator all calculate during EXECUTE, every time; their answers are simply not selected when they are not wanted. **This diagram is dense on purpose: it is the design itself, not a simplification. Zoom in to read the signal names.**

[View this diagram full size](#) (opens the image in the project repository)

Or zoom to 300 % in any reader: stored here at about 580 dpi · the same image is in the submission as `soc-architecture.png`



Instruction class	Cycles
Branch (taken or not)	3
Store (SB/SH/SW)	4
ALU-reg, ALU-imm, LUI, AUIPC, MUL*, CRC*, JAL, JALR, FENCE, ECALL/EBREAK, illegal	4
Load (all five widths)	5

This table is the control unit's own next-state logic. It was verified by 53 directed transition checks, and by driving all 704 possible instruction encodings through the real state machine.

Figure 6: Control-unit FSM: six states, 3-bit binary encoding, and the resulting cycle count per instruction class. `rst_i` is synchronous and additionally gates every combinational control output directly, so an in-flight store's write enable drops in the *same* cycle reset asserts rather than one cycle later, confirmed at SoC level by injecting reset at 40 different points of a running program and checking that no data-memory word changed across the reset edge.

Table 3: Every multiplexer in the datapath, what it can choose between, and which signal makes the choice. The control unit drives all of these selects. The program-counter select also depends on `branch_taken` in the same cycle. This is deliberate, and it is what lets a branch finish in three cycles.

Mux (instance)	Select	Sources	Purpose
u_muxa	alu_src_a_sel	rs1_data pc	ALU operand A; pc for AUIPC and branch/JAL target math
u_muxb	alu_src_b_sel	rs2_data imm	ALU operand B; immediate for every I/S/B/U/J format
u_muxaddr	addr_src_sel	pc alu_result	Shared memory address: pc during fetch, effective address during EXECUTE/MEMORY, and through WRITEBACK for loads
u_muxpc	pc_src_sel	pc_plus_4 alu_result	Next PC; target when a branch is taken or on JAL/JALR
u_wbmux	wb_src_sel[2:0]	alu_result mult_result crc_result lsu_load_data pc_plus_4	Register-file write data
n/a	alu_op[3:0]	11 operations, Block Guide Table 9 (4'h0 PASS_B ... 4'hA SLTU)	ALU function select
n/a	op_size[2:0]	funct3, republished by the control unit	Drives <code>branch_comparator</code> , <code>multiplier</code> , <code>crc_unit</code> and <code>lsu</code> from one source

One timing detail: a load that reads from the program memory

The Block Guide (§4.2) permits a LW to read from a program-memory address. Programs use this to read constants that they store between their instructions. The competition firmware does it twice. It reads its CRC test data from 0x004003F8 and its copy-loop source data from 0x00400400.

The two memories behave differently, and this matters. The data memory has an output register, so it holds the word it read. The program memory has no output register. Its output follows its enable immediately.

If the read enable went low at the start of WRITEBACK, the program memory output would fall to zero in that same cycle. The register file samples the value in exactly that cycle. The load would then write zero, and nothing would report an error.

The fix is to keep the read enable and the address selection at their EXECUTE values through WRITEBACK, but only for loads. This costs no extra register. It has no effect on data-memory loads.

Stores are different. A store has no WRITEBACK cycle, and its write enable must go low before the address changes. The write enable is therefore limited to MEMORY. This difference between loads and stores is deliberate. Validation stage 7 confirms the behaviour on the complete system.

4 Modules and Testbenches

In one paragraph. On the ChipInventor canvas the processor is **17 block definitions**. These are placed as **20 instances** and joined by **72 connections**, which form **45 nets**. The top level has exactly **2 pins**. Each block is a unit that can be tested on its own, and each has its own directed testbench.

This section gives four things: the interface and role of every block, the testbench that covers it, how the blocks are connected on the platform, and the twelve automatic gates that must pass before anything is pasted into ChipInventor.

Evidence	Measured	What it demonstrates
Block definitions on the canvas	17	Each one verified on its own
Instances, and nets between them	20 / 45	72 connections, 2 top-level pins
Reference regression, block level	7,384	15 testbenches, 0 failures
with the gate-level testbench	8,285	16 testbenches, needs Yosys
Platform suite, one pasted file	15,281	5 suites, 0 failures
Automatic gates before submission	12	All pass, see Listing 1
Mirror equivalence, cycle by cycle	410,400	0 mismatches against the reference design
Competition firmware verdict	PASS	x4 = 0x00000000 after 991 cycles

The two suites are independent. The reference regression tests the design as sixteen separate files. The platform suite tests the single file that ChipInventor actually runs. A fault would have to appear in both to pass unnoticed.

4.1 Per-module implementation and its testbench

Table 4: The 17 canvas blocks: interface width, role, and the testbench that verifies each. “Checks” is the count from the standalone regression run reproduced for this report; blocks marked *glue* are covered by suite S0.11 of the platform testbench and indirectly by every integration test.

Table 4a Control and instruction-issue blocks

Block	In/Out	Role	Testbench	Checks
control_unit	4/15	6-state FSM; instruction-class decode on the full {opcode, funct3, funct7} tuple; emits every control signal	tb_control_unit: 704-encoding sweep, 53 transition-table checks, mid-instruction reset abort	1461
pc_reg	4/1	Program counter; reset vector 0x00400000; bits [1:0] forced to 0 on every load	tb_core, tb_soc, tb_reset; S1.2	n/a
pc_incrementer	1/1	Dedicated PC+4 adder, deliberately not the shared ALU	S0.11 <i>glue</i> ; exercised by every program	n/a
ir_reg	4/1	Instruction register; written once per instruction, at the FETCH→DECODE edge	S0.11 <i>glue</i> ; tb_core	n/a
ir_fields	1/3	Extracts rs1/rs2/rd addresses; exists because the canvas cannot slice a bus (§4.2)	S0.11 <i>glue</i>	n/a
immediate_generator	1/1	I/S/B/U/J immediate extraction and sign extension, purely combinational	tb_immediate_generator: checked against an independent Python bit-slicer	34
register_file	7/2	32 × 32-bit GPRs; x0 hardwired to zero; 2 asynchronous reads, 1 synchronous write	tb_register_file: x0 write-guard, full round trip, reset	36
mux2_32 (×4)	3/1	Generic 2-to-1 32-bit multiplexer; four instances, per Table 3	S0.11 <i>glue</i>	n/a

Table 4b Execution, memory and interconnect blocks

Block	In/Out	Role	Testbench	Checks
alu	3/1	11 operations exactly per Block Guide Table 9; shift amounts masked to b[4:0]; safe default	tb_alu: directed per-op corners (0, ±1, INT_MIN/MAX, shift ≥32) + 1,100 randomised	1128
branch_comparator	3/1	The 6 RV32I branch conditions, signed/unsigned aware; reserved funct3 defaults to not-taken	tb_branch_comparator: all 6 in both directions, signed/unsigned boundary	24
multiplier	3/1	MUL/MULH/MULHSU/MULHU, combinational 64-bit product, Table 10	tb_multiplier: 2,198 vs an independent 64-bit reference, + 4 truth-table checks of the sign/half formulas	2202
crc_unit	3/1	CRC-16/CCITT-FALSE over 8/16/32-bit inputs as three constant-width XOR networks plus an output mux, per Table 11	tb_crc_unit: independent Python bit-serial model, the standard 0x29B1 check value, the firmware’s 0x1E82 anchor	322
lsu	5/3	Byte-lane placement, sign/zero extension, 4-bit byte-write strobe generation	tb_lsu: reproduces the Block Guide’s own worked examples exactly, plus a full alignment sweep	26
wb_mux_32	6/1	5-source writeback multiplexer (ALU, multiplier, CRC, load data, PC+4)	S0.11 <i>glue</i>	n/a
address_decoder	6/5	Memory-mapped routing per Table 13; we never reaches the ROM; bw reaches only the SRAM	tb_address_decoder: boundary sweep and out-of-range store asserts zero write enable	12
imem	2/1	742-word instruction/constant ROM, combinational read, oe-gated	tb_imem: oe gating; out-of-array reads return 0, not an alias	7
dmem	7/1	Synchronous, byte-writable data SRAM; DMEM_WORDS = 2048 (the full 8 kB) on the canvas	tb_dmem: byte lanes, true storage, write-wins, implemented-size boundary	17

Four more testbenches test the assembled system instead of one block. They are `tb_core` (29 checks on the core with the decoder and both memories), `tb_soc` (14 checks on the two-pin top level, including a load from the program memory), `tb_reset` (2,002 checks, with reset applied at 40 different points of a running program) and `tb_firmware` (70 checks that reach all 47 instructions through 58 signature tests).

A sixteenth testbench, `tb_gatelevel`, compares the design against a synthesised netlist cycle by cycle for 900 cycles. It needs the Yosys tool. If that tool or its netlist is absent, the suite skips this testbench. It does not fail it. The run shown in §6 is such a run.

4.2 SoC-level interconnection on the ChipInventor platform

The canvas imposes two constraints that shaped the block decomposition, and both are worth stating because they explain structure that would otherwise look redundant.

The canvas cannot cut a bus into pieces. Each connection joins one complete port to another complete port of the same width. An exported project contains no partial connection at all. You therefore cannot draw `ir[19:15]` as a wire.

This is why `ir_fields` is a block of its own. It takes the instruction word and publishes the three register numbers as separate ports. For the same reason the design never extracts `funct3` twice. The control unit publishes it once as `op_size`, and the branch comparator, the multiplier, the CRC unit and the load-store unit all use that one output. One source cannot disagree with itself.

Macros leak between blocks. The platform joins the source of every block into one file. A `‘define` in one block is therefore visible in all the others. Every block here uses `localparam` instead, which belongs to one module only.

Gate 9 checks this automatically. It also rejects four other constructs: `‘ifdef`, `‘include`, `$readmemh` and `initial` blocks. Each of these fails *quietly* on a platform that has no file system.

4.2.1 A two-pin top level, and how observability was preserved Block Guide Table 5 permits exactly two pins, and both are inputs. The chip therefore has no data output. This creates a real danger. A synthesis tool removes any logic that cannot reach an output, so it can delete the whole design. This is not a theory. During development Yosys did exactly that and reported **0 cells**.

The defence is to mark the storage elements with `(* keep *)`. Three are marked: the data memory array, the register file array, and the state register of the control unit. Anything that feeds a kept register also survives, so these three marks hold the whole datapath in place.

One detail about the platform matters here. The platform rebuilds each module header from its own input and output form fields. A mark on a *port* declaration is therefore lost in the export, but a mark on an *internal* declaration is kept. Gate 11 lists the three marks that were lost. It lists them as notes, not failures, because the three kept arrays already hold those paths.

The cell count proves the defence worked. Synthesis produced **17,708 cells and 1,393 flip-flops**. The architecture needs about 1,315 flip-flops: 992 for the register file (31 registers of 32 bits; `x0` is hardwired and needs none), 256 for the data memory in this build, 32 for the program counter, 32 for the instruction register and 3 for the state. Every storage element is present, and nothing was removed.

Table 5: What a complete RVBL-2 project on the canvas contains. These counts are recounted automatically from the design before every submission.

Canvas element	Count
Block definitions to create	17
Block instances to place	20
Input pins (<code>clk_i</code> , <code>rst_i</code>)	2
Nets	45
Individual connections to draw	72
Internal wires in the export	41
Module definitions in the export	18
ROM words	742

Two control-unit outputs, `state_o` and `sys_event_o`, are deliberately left unconnected: the top level has only the two pins Block Guide Table 5 mandates, so there is nowhere for them to go. The testbench observes them through hierarchical references instead. This is why three different net counts appear in this report, and all three are correct: the canvas checklist has **45** rows, the exported netlist has **43** nets because those two have no destination, and the top module declares **41** internal wires because two of the 43 are the input pins themselves.

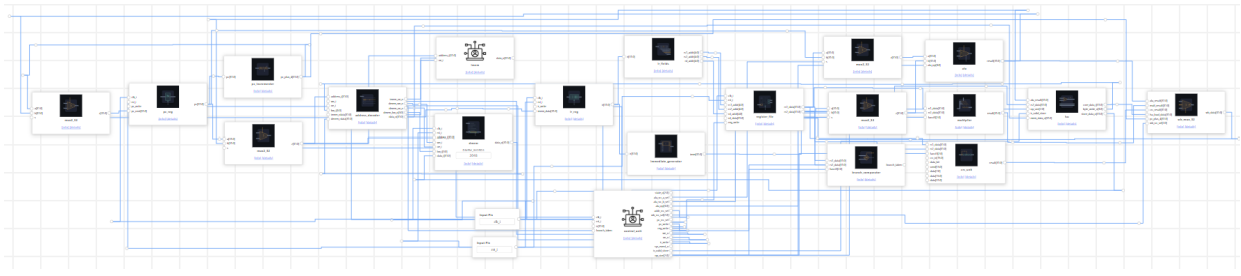


Figure 7: The processor as drawn on the ChipInventor canvas. Each white card is one block instance. Each blue line is one wire. There are 20 instances and 72 connections, which together form 45 nets. The two small cards at the bottom left are the only input pins, `clk_i` and `rst_i`.

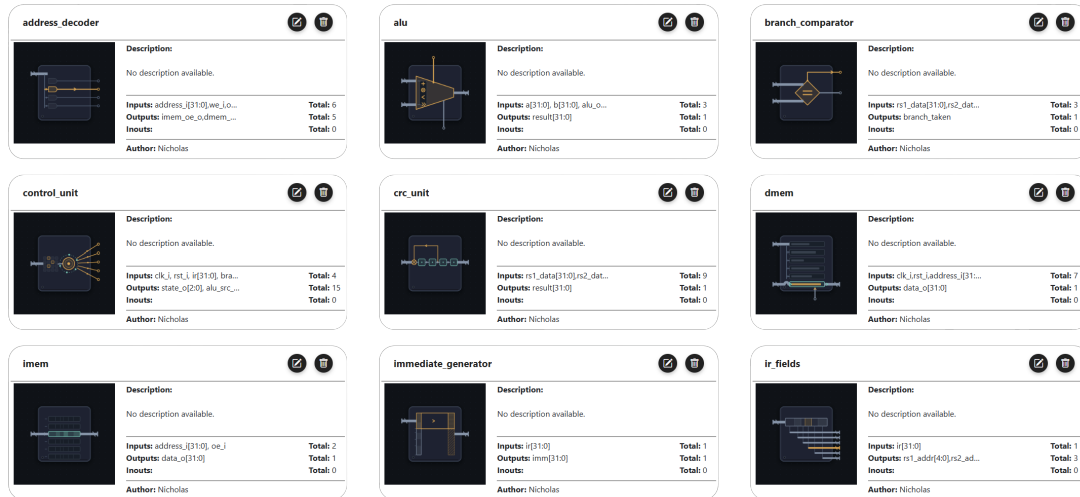


Figure 8: The block library on the platform, showing 9 of the 17 blocks. Each card gives the block name, the icon the platform generates, and the number of inputs, outputs and bidirectional ports. No block has a bidirectional port. The port counts here match the interface column of Table 4 exactly, which confirms that what was entered on the platform is what was verified.

4.3 The platform testbench and the twelve verification gates

The platform runs a testbench only at project level. It cannot run one testbench per block. We therefore combined our sixteen separate testbenches into one file. That file is what we paste into the platform's testbench field.

The combined testbench creates the project as `top dut`, connects the clock and the reset, and then reads the internal signals through hierarchical names. This is the only way to observe a design that has no data outputs.

Two changes were necessary. The platform has no file system, so the tests cannot read a file of expected values. Each of those files became a reference function inside the testbench instead. The functions also sweep more values than the fixed files did.

The combined testbench runs **15,281 checks** in five suites:

- **S0**, about 9,900 checks: each block on its own.
- **S1**, about 1,500 checks: the control state machine.
- **S2A**: the official validation firmware.
- **S2B**, about 80 checks: our own coverage program.
- **S3**, about 3,700 checks: reset applied during a running program.

Before anything is pasted, a script runs twelve gates on a local machine. Listing 1 is that script's own output, copied into this report without editing.

```

=== 1/12 Official validation firmware is intact ===
OK   official firmware: 259 words, 0x00400000..0x00400408, contiguous, sha256 pinned
=== 2/12 Supplementary firmware image is current ===
OK   position-independent: 483 words identical at 0x00400000 and 0x00400800

```

```

=== 3/12 Generated ROM matches both firmware images ===
OK imem.v: 742 words - validation_firmware.hex @ word 0 (259), ci_prog.hex @ word 512 (483)
=== 4/12 Testbench constants match the firmware ===
OK official firmware anchors still point at the right instructions (4)
=== 5/12 Mirror equivalence: ci_top.v vs the verified original ===
OK 410400 passed, 0 failed
=== 6/12 Combined suite against the local mirror ===
OK 15281 passed, 0 failed
=== 7/12 OFFICIAL VALIDATION FIRMWARE VERDICT ===
OK Firmware settled after 991 cycles at pc=004003ec (all_good)
OK x4 = 00000000 - the official firmware PASSES on this core
=== 8/12 Mock IMEM drives the core (this is what OpenLane synthesises) ===
OK mock ROM runs on the core: x5=10, x6=5, x7=15
=== 9/12 Blocks are free of constructs the platform cannot resolve ===
OK no ifdef / include / file I/O in any block
OK no initial blocks in any block (nothing leaks into synthesis)
=== 10/12 Schematic layout is clean ===
OK canvas 3923x1242, 20 blocks, 46 routed wires, 12 columns
=== 11/12 Exported netlist matches the verified design ===
OK all 17 block sources match blocks/*.v
OK ROM is bit-identical to blocks/imem.v (742 words)
OK 20 instances, same block mix as ci_top.v
OK netlist is graph-identical to ci_top.v (43 nets; unconnected in the export:
  control_unit.state_o, control_unit.sys_event_o)
OK dmem DMEM_WORDS = 2048
=== 12/12 Combined suite against the EXPORTED netlist ===
OK 15281 passed, 0 failed on the exported netlist
OK official firmware PASSES on the exported netlist
=====
ALL CHECKS PASSED - safe to paste into ChipInventor
=====

```

Listing 1: The twelve verification gates, exactly as the script printed them. Gates 11 and 12 activate only once the project has been exported from the platform into the repository, which it has.

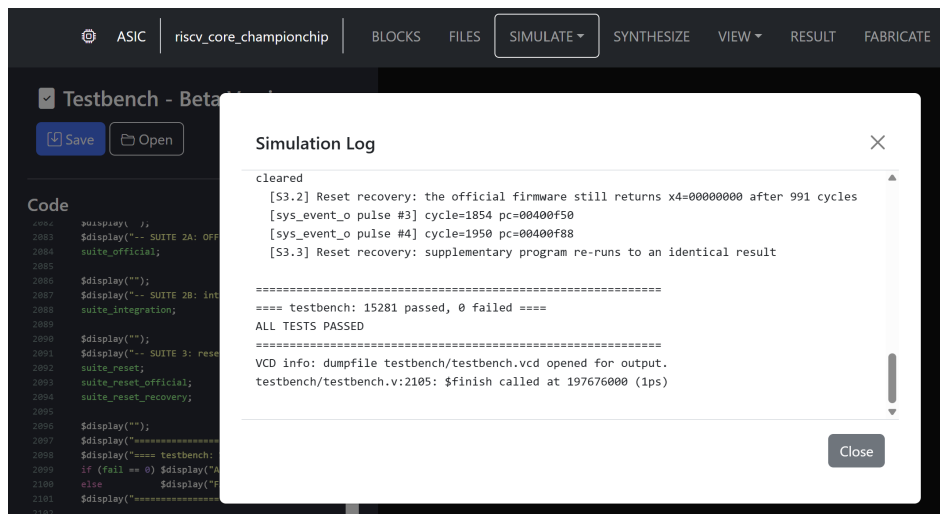


Figure 9: The same testbench running *on the platform*, not on a local simulator. The platform’s own Simulation Log reports 15281 passed, 0 failed and ALL TESTS PASSED. The visible lines above the total are the last three checks of the reset-stress suite, including the re-run that returns `x4=00000000` after 991 cycles. This is the result that matters: the design passes where it will be built.

4.4 Synthesis and place-and-route on the platform

ChipInventor’s own OpenLane installation produced the layout in §5. It used the configuration submitted with this project, and it worked on the design that the platform itself generated from the canvas.

Two independent facts prove that the platform built this design and not some other one. First, gate 11 shows that the exported netlist has the same structure as the verified one. Second, all seventeen blocks in the file the platform generated were compared against the submitted source. Every one was identical. The top module carries 41 wires with the same names, the same widths and the same order.

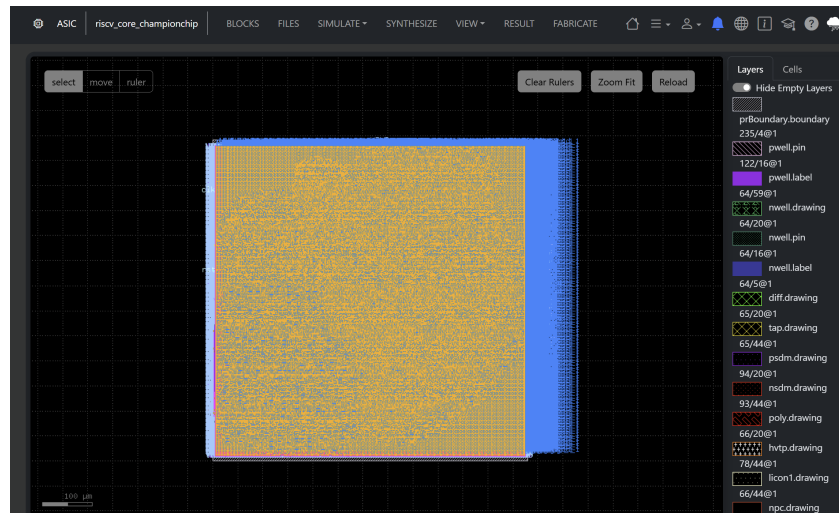


Figure 10: The finished layout opened in the ChipInventor layout viewer. The panel on the right lists the Sky130 mask layers that the GDSII file contains, from the wells and diffusion up through the metal layers. The scale bar at the bottom left reads 100 μm . This is the platform showing its own result, not a local copy.

5 OpenLane Results

In one paragraph. The design was taken from Verilog to GDSII by OpenLane on the ChipInventor platform, targeting SkyWater sky130A with the `sky130_fd_sc_hd` standard-cell library and *no* hard macros. The flow completed with zero design-rule violations, zero routing violations, a layout-versus-schematic match that is unique, and positive setup and hold margin at all three timing corners. Table 6 is the area and density result the Submission Guide asks for; Figure 12 is the final GDSII.

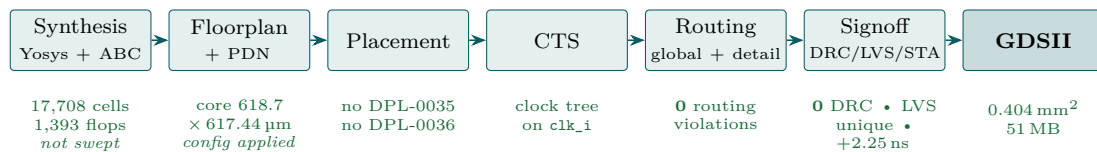


Figure 11: The OpenLane flow, and the check applied at each stage. “Did the flow finish” is the wrong question, for two reasons. First, a two-pin design can be reduced to nothing and still pass every later stage. Second, OpenLane does *not* stop for a timing violation. It returns a complete layout that simply misses its clock target. We therefore checked each gate above on its own. Every result in Table 7 was read from the report file the tool wrote.

5.1 What was synthesised, stated precisely

Two reductions apply to the place-and-route netlist, both on the organisers’ instruction, and both are visible in the run’s own logs rather than only in our source. They are stated here rather than buried, because a reviewer who finds them unannounced would be right to discount everything else.

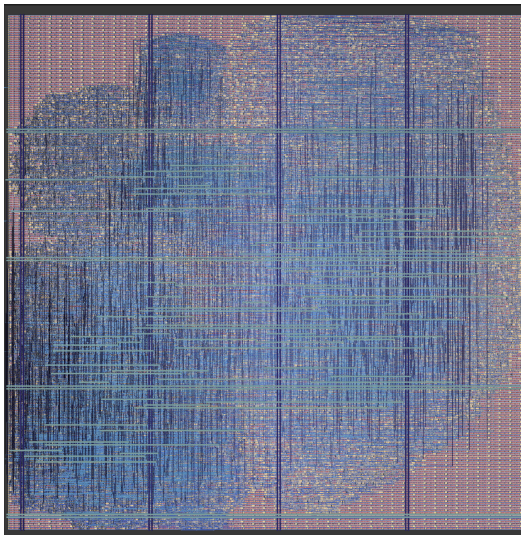
The full 8 kB window needs 65,536 flip-flops. Measured locally, that is 2.43 mm^2 of standard cells, about 95 % of the design, and it does not fit in this die. The functional design on the canvas keeps all 2,048 words, and gate 11 checks that on the exported netlist.

Table 6: Total area and density after physical implementation (Submission Guide §5). Every figure is taken from the run’s own metrics report.

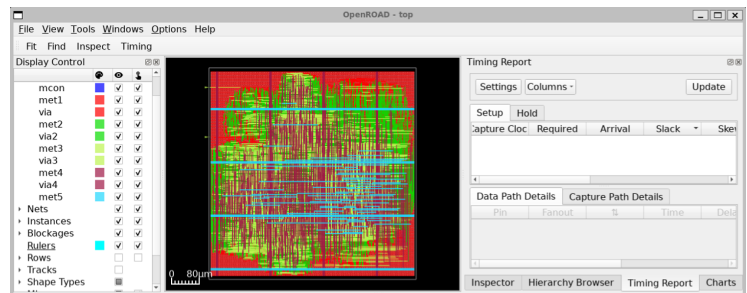
Metric	Value
Die dimensions	$630.15 \times 640.87 \mu\text{m}$
Total die area	0.4038 mm^2
Core dimensions	$618.70 \times 617.44 \mu\text{m}$
Core area	$382,010 \mu\text{m}^2$
Standard-cell area	$191,649 \mu\text{m}^2$
Final utilisation (density)	52.28 %
Cell density	$47,786 \text{ cells/mm}^2$
Standard cells	17,708
Flip-flops	1,393
Total placed instances	51,267
decap / fill / well-tap	17,132 / 8,940 / 5,381
antenna diodes	516
Total wirelength	$869,840 \mu\text{m}$ (0.87 m)
Vias	151,445

Table 7: Signoff results. Every row is read from the run’s own report file; none is estimated.

Signoff check	Result
Detailed-routing violations	0
Shorts / spacing / off-grid	0 / 0 / 0
Magic DRC	0
KLayout DRC	0
Magic ↔ KLayout XOR	no differences
LVS	match uniquely
devices / nets, both sides	19,544 / 19,302
Residual antenna	86 pins / 60 nets
<i>Multi-corner static timing</i>	
Clock constraint	33 ns (30.303 MHz)
Setup slack, fast	+3.83 ns
Setup slack, typical	+3.08 ns
Setup slack, slow	+2.25 ns
Hold slack, worst	+0.11 ns
Total negative slack	0.00 ns
<i>implied period,</i>	30.75 ns ($\approx 32.5 \text{ MHz}$)
<i>slow</i>	
Technology	sky130A, sky130_fd_sc_hd
OpenLane / PDK	3876562d / bdc9412b
Flow runtime	54 m 53 s



(a) the finished layout



(b) the same layout, by metal layer

Figure 12: The signed-off layout. (a) The complete die. It measures $630.15 \times 640.87 \mu\text{m}$, or 0.404 mm^2 , and is 52.28 % full. It contains 17,708 standard cells. Two independent checkers found zero design-rule violations, and the layout-versus-schematic check matched the netlist uniquely. (b) The same layout in the OpenROAD viewer, with the metal layers coloured: red is **met1**, green is **met2** and **met3**, and light blue is **met5**. The wiring is spread evenly, with no dense spot. This agrees with the zero routing violations in Table 7. The timing panel on the right lists no failing path.

Table 8: The two synthesis reductions, the evidence for each in the submitted run, and why each is correct rather than convenient.

Reduction	Evidence in the submitted run	Why it is sound
DMEM_WORDS = 8 (32 bytes instantiated, not the full 8 kB)	The synthesis log records Parameter DMEM_WORDS = 8. The cell report agrees: it counts 256 flip-flops, which is 8 words of 32 bits.	A guard keeps every address in the 8 kB window defined. Past the end of the array, reads give 0 and writes do nothing. No address can alias onto a real word. Block Guide §4.3 permits a smaller array. The size is the smallest that works, not an arbitrary choice: 1, 2 and 4 words all <i>fail</i> the firmware (<code>x4 = ffffffff</code>); 8 <i>passes</i> . The firmware uses 28 bytes.
3-instruction mock ROM in place of the 742-word firmware ROM	The file the tool actually read declares the program memory with three entries instead of 742. That file is generated from the exported netlist by a script. Nobody edits it by hand.	The firmware repository gives this advice itself: a 742-word case block “can generate unnecessary cells”. The mock has the same module name and the same ports, so nothing on the canvas changes. It cannot go stale unnoticed: gate 8 builds the <i>whole core</i> against the mock and runs the repository’s own three-instruction program to <code>x7 = 15</code> .

5.2 Three results worth reading

1. Instance order is an area knob, worth 12 %. The platform’s generated `hdl.v` and the local reproduction are identical in every module body, in the top-level wire list, and in the resolved OpenLane environment. They differ in exactly one thing: where the `imem` block sits in the instance list. Moving that single instantiation, changing *no logic whatsoever*, reproduces the platform’s synthesis to the byte. Yosys derives modules in instantiation order, which changes internal ID allocation, which changes the visit order of order-sensitive passes (`opt_merge`, `share`, `fsm`, `opt_muxtree`). Two consequences carry forward: canvas block order is a free area knob, and *a cell-count difference between two runs is not by itself evidence that the RTL changed*.

	imem 5th	imem last
Pre-ABC cells	17,073	19,080
<code>\$_XNOR_ / \$_XOR_</code>	941 / 2,644	1,386 / 3,064
Mapped cells	15,524	17,708
Cell area (μm^2)	171,332	191,649

2. Heuristic antenna-diode insertion cost 19.5 % of the die for nothing. `RUN_HEURISTIC_DIODE_INSERTION` places a diode pre-emptively on nearly every net. It inserted **9,838** diodes, a 63 % cell-count increase applied *after* the floorplan had already been sized, and left *exactly the same* residual violating nets at signoff as targeted repair with **325**. Disabling it shrank the die from 666×676 to $596 \times 607 \mu\text{m}$ (−19.5 %), returned $\approx 1.9 \text{ ns}$ of setup slack, and turned a GRT-0119 routing-congestion failure into a clean route. This is why `config.json` sets it to 0 explicitly.

3. Reported frequency versus achieved margin, both stated honestly. The run reports **30.303 MHz**, but that figure only ever echoes the constraint: OpenLane clamps positive slack to zero (`wns = 0.0`) before computing `suggested_clock_frequency`. The design actually closes with 2.25 ns to spare at the slow corner, i.e. at $\approx 30.75 \text{ ns}$, or **$\approx 32.5 \text{ MHz}$** on this run. Two caveats we state rather than omit: slack varied by 1.6 ns run-to-run on identical inputs, and the local run series measured a required period of 32.36 ns regardless of the target. **Treat 33 ns as the guaranteed constraint and $\approx 32.5 \text{ MHz}$ as a single-run demonstration, not a characterised f_{max} .**

5.3 Residual issues, reported rather than papered over

- **Residual antenna violations: 86 pins on 60 nets**, worst partial/required ratio **$8.69 \times$** the 400 limit, mostly on `met3`. Enabling heuristic diode insertion does *not* reduce them, and costs 19.5 % of

the die, so they are recorded rather than papered over. They do not affect DRC, LVS or the routing result.

- **Max-slew and max-fanout warnings at the typical corner.** The default `MAX_FANOUT_CONSTRAINT` is 10; the offenders are high-fanout control and operand-broadcast nets. Real signoff warnings, but not what limits $f_{\max}$.
- **IR-drop analysis is approximate** (`VSRC_LOC_FILES` unset), which is expected for a design that is not a top-level chip integration.
- **No timing-annotated simulation was run after layout.** We did compare the design against the *post-synthesis* netlist for 900 cycles, and the results were identical. Signoff timing analysis also reports positive margin at every corner. However, we did not keep the 88 MB timing file, and we did not run a gate-level simulation with it. This is the one gap in the flow, and we state it as such.

6 Firmware Demo

In one paragraph. The official ChampionCHIP validation firmware runs on this core. It returns `x4 = 0x00000000`, which is its own PASS value. It reaches that result after **991 clock cycles** and then stops in its `all_good` loop. The firmware contains nine test stages, one after the other. **All nine complete.** The firmware does not use `ECALL` or `EBREAK` to report that it has finished. It ends with `j .`, an instruction that jumps to itself. The testbench therefore watches the program counter. When the program counter stops at one of the two exit loops, the firmware has finished, and the testbench reads the result from register `x4`.

Every stage that fails jumps to the same error label. A single failure would therefore hide which stage caused it. To prevent this, the testbench records the address the jump came from. That address identifies the stage.

Table 9: All nine validation stages of the official firmware, and the evidence each completed. PC ranges are the branch-source attribution boundaries the testbench uses to name a failing stage. Because the stages are sequential and each ends in a conditional branch to `_error`, reaching `all_good` at `0x004003E8` is proof that *every* stage passed. There is no path to it that skips one. The stage names and their address ranges are taken from the competition firmware’s own source listing.

#	Stage	PC range	What it exercises	Result
1	<code>_rv32i_alu_test</code>	00400000-00400118	LUI, AUIPC, ADDI/ADD/SUB, all logic ops, SLL/SRL/SRA including SRAI with a shift amount requiring <code>b[4:0]</code> masking, SLT/SLTU at the signed/unsigned boundary	PASS
2	<code>_rv32i_lsu_test</code>	0040011C-00400180	<code>.bss</code> base derived from the PC (<code>auipc s0,0xfc10 → 0x10010000</code>); SW/SH/SB then LW/LH/LHU/LB/LBU: all eight width/offset combinations	PASS
3	<code>_rv32i_ctrl_test</code>	00400184-0040020C	All six branch conditions in both directions, including negative operands (<code>-10</code>) that separate signed from unsigned comparison; JAL and JALR control transfer	PASS
4	<code>_rv32i_reg_test</code>	00400210-00400234	Register-file integrity: <code>addi zero, zero, 1</code> followed by <code>bne zero, zero, _error</code> probes the <code>x0</code> hardwiring; a value is propagated through <code>x5→x6→x7→x31</code>	PASS
5	<code>_rvbl2_mult_test</code>	00400238-00400294	All four <code>Zmmul</code> instructions with operands chosen to separate them: 100000×2 , an 8×10^9 product exercising the upper half, and a <code>MULHSU</code> case that distinguishes signed $\times$ unsigned from both alternatives	PASS
6	<code>_rvbl2_crc_test</code>	00400298-00400348	CRCB/CRCH/CRCW chained to the firmware’s own expected <code>0x1E82</code> . This stage found a real bug: the CRC block took its two operands the wrong way round. Earlier tests could not see it, because CRCH gives the same answer either way	PASS
7	<code>_rvbl2_crc_mem_test</code>	0040034C-00400374	LW of <code>.rodata</code> located in the instruction ROM at <code>0x004003F8</code> , then CRCW over it: the exact case the datapath’s extended <code>oe_o/address</code> scoping exists for	PASS
8	<code>_rvbl2_arith_integ_test</code>	00400378-00400390	Integration: $(A + B) \times (A - B)$ across ADD, SUB and MUL in sequence, checking that results feed correctly between instruction classes	PASS
9	<code>_rvbl2_mem_transfer_test</code>	00400394-004003E4	A three-word copy loop reading its source from the ROM at <code>0x00400400</code> and writing to data memory, then reading the destination back: loads, stores, pointer arithmetic and a backward branch together	PASS
Verdict: <code>all_good</code> at <code>0x004003E8: li x4, 0 ; j .</code>				9/9

6.1 Testbench log output proving the run reached the end without errors

```

=====
RVBL-2 combined verification suite
RV32I + Zmmul + Xicrc, 47 instructions
=====
-- SUITE 0: unit tests (core held in reset) -----
CHECK-VALUE GATE: chained CRCB("123456789") = 0x29b1 (expected 0x29B1) PASS
[S0.8] CRC-16/CCITT-FALSE: KAT gate, firmware 0x1E82 anchor, corners, 600 randomised
-- SUITE 1: control unit -----
[S1.1] Control decode sweep: 704 encodings through the real FSM
-- SUITE 2A: OFFICIAL VALIDATION FIRMWARE -----
Firmware settled after 991 cycles at pc=004003ec (all_good)
*****
* OFFICIAL VALIDATION FIRMWARE: x4 = 00000000 PASS *
*****
[S2A] Official validation firmware: 991 cycles, verdict x4=00000000
-- SUITE 2B: integration, supplementary coverage program -----
[sys_event_o pulse #1] cycle=1854 pc=00400f50
[sys_event_o pulse #2] cycle=1950 pc=00400f88
Ran 6000 cycles, observed 2 sys_event_o pulses (expect 2: EBREAK then ECALL)
Signature-based instruction tests: 58/58 passed
-- SUITE 3: reset stress -----
[S3.1] Reset injected at 40 points of the official firmware: writes aborted, PC/FSM/GPRs cleared
[S3.2] Reset recovery: the official firmware still returns x4=00000000 after 991 cycles
=====
==== testbench: 15281 passed, 0 failed ====
ALL TESTS PASSED
=====

```

Listing 2: The platform testbench's own output, verbatim. SUITE 2A is the official validation firmware; the surrounding suites are the project's own coverage. SUITE 3 re-runs the official firmware after injecting reset at 40 different points of its execution and confirms it still returns x4=00000000.

```

=====
Unit-level testbenches
=====
PASS      tb_alu      ===== tb_alu: 1128 passed, 0 failed =====
PASS      tb_branch_comparator ===== tb_branch_comparator: 24 passed, 0 failed =====
PASS      tb_immediate_generator ===== tb_immediate_generator: 34 passed, 0 failed =====
PASS      tb_register_file ===== tb_register_file: 36 passed, 0 failed =====
PASS      tb_multiplier ===== tb_multiplier functional: 2198 passed, 0 failed =====
PASS      tb_crc_unit  ===== tb_crc_unit: 322 passed, 0 failed =====
PASS      tb_lsu       ===== tb_lsu: 26 passed, 0 failed =====
PASS      tb_imem      ===== tb_imem: 7 passed, 0 failed =====
PASS      tb_dmem      ===== tb_dmem: 17 passed, 0 failed =====
PASS      tb_address_decoder ===== tb_address_decoder: 12 passed, 0 failed =====
PASS      tb_control_unit ===== tb_control_unit: 1461 passed, 0 failed =====
=====
Integration-level testbenches
=====
PASS      tb_core      ===== tb_core: 29 passed, 0 failed =====
PASS      tb_soc       ===== tb_soc: 14 passed, 0 failed =====
PASS      tb_reset     ===== tb_reset: 2002 passed, 0 failed (40 injection points) =====
PASS      tb_firmware  ===== tb_firmware: 70 passed, 0 failed =====
TB_FIRMWARE: ALL TESTS PASSED (47/47 instructions verified)
=====
SUMMARY: 15/15 testbenches passed, 7384 individual checks
=====

```

Listing 3: The reference regression, run block by block against the reference design, and copied into this report without editing. `tb_gatelevel` needs a synthesised netlist. When that netlist is absent the suite skips this testbench and does not fail it. With it, the suite is 16 of 16 and 8,285 checks.

6.2 Waveform evidence

A waveform is a picture of the simulation. Time runs from left to right. Each row is one signal. A single-bit signal is drawn as a square wave, high or low. A multi-bit signal is drawn as a stretched hexagon with its value written inside, and the hexagon pinches at each point where the value changes. The clock row gives the scale: one full high-low pair is one clock cycle, which is 10 ns here.

The two figures below come from one simulation run. That run executes the competition test program on the netlist the platform exported.

Official ChampionCHIP validation firmware: x4 = 0x00000000 (PASS) after 991 cycles

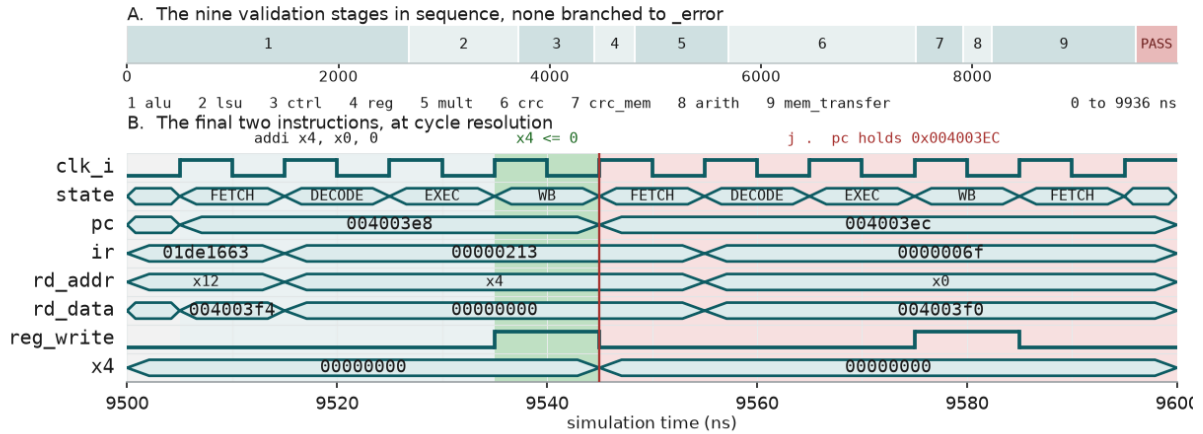


Figure 13: The competition test program completing. Panel A is a summary of the whole run. Each numbered band is one of the nine test stages, drawn to scale in time, and the red band at the right is the result. Panel B magnifies the last 100 ns. The core executes `addi x4, x0, 0`, which writes the PASS value into the verdict register: the `rd_addr` row shows `x4`, the `rd_data` row shows `00000000`, and `reg_write` goes high for one cycle in WB. The red line marks that write taking effect. The core then executes `j .`, a jump to itself, so the `pc` row stops at `004003ec` and never moves again.

What Figure 13 proves

The program only reaches `addi x4, x0, 0` if every one of the nine stages passed. Each stage ends with a branch to a shared error label, and that label writes `-1` instead. The waveform shows the PASS write happening and the program counter stopping, so no stage branched away. Panel A shows the nine stages in sequence, which is the evidence Submission Guide §6 asks for.

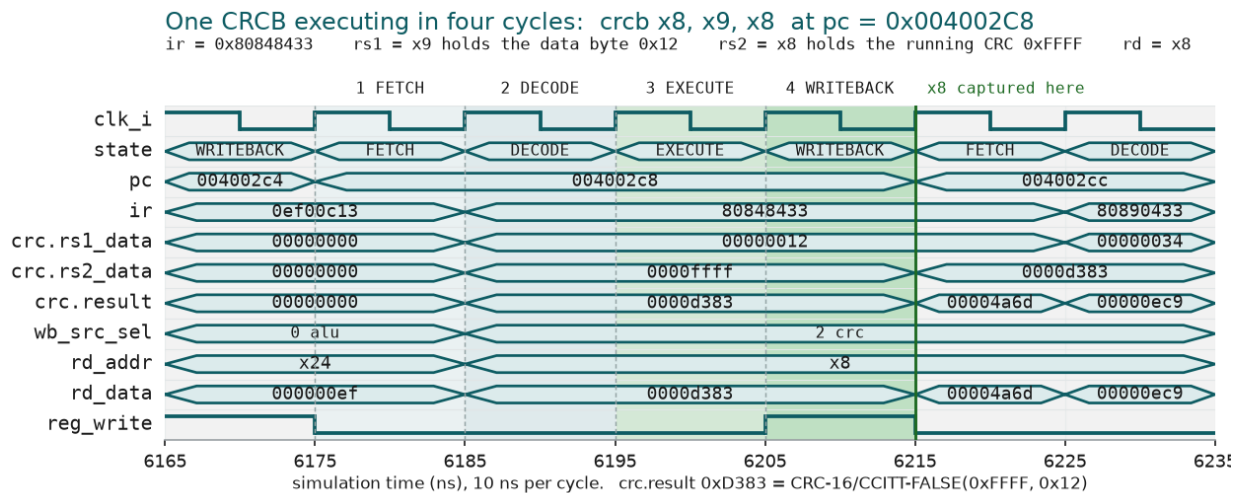


Figure 14: One CRCB instruction of the custom Xicrc extension, executing in four clock cycles. The `state` row steps `FETCH` → `DECODE` → `EXECUTE` → `WRITEBACK`, which matches the four-cycle path in Figure 6. The operands are visible: `crc.rs1_data` carries the data byte `0x12` and `crc.rs2_data` carries the running checksum `0xFFFF`. The answer `0xD383` appears on `crc.result` during `DECODE` and stays steady, because the CRC block is combinational and has no clock. `wb_src_sel` reads `2 crc`, so the writeback multiplexer selects the CRC port. The green line is the `WRITEBACK` → `FETCH` edge, where register `x8` captures the answer.

Three things in Figure 14 are worth reading carefully, because together they show the whole datapath working on one instruction.

1. **The result is ready before it is needed.** `crc.result` settles during `DECODE`, one full cycle before `WRITEBACK` uses it. This is expected: the CRC block is pure logic, so its output follows its

inputs with no clock. EXECUTE therefore spends its cycle on a value that is already correct. The same is true of the ALU and the multiplier, which is why this processor needs no result register between states.

2. **The right unit is selected.** Four units compute an answer at the same time from the same two operands. `wb_src_sel` chooses which answer reaches the register file. It reads 2 `crc` here, and it would read 0 `alu` for an addition.
3. **The answer is chained.** Immediately after the green edge, `crc.rs2_data` changes from 0000ffff to 0000d383. The next CRCB has read back the value this one just wrote. A checksum is built one byte at a time, so this hand-off is the instruction doing its job.

An independent check confirms the number. The standard CRC-16/CCITT-FALSE algorithm, applied to the byte 0x12 with the start value 0xFFFF, gives 0xD383. That is the value on `crc.result`.

6.3 Verification beyond the verdict register

The verdict register alone is not sufficient proof. The firmware checks its own work with branches, and those branches read the same registers the firmware has just written. A core could therefore calculate every comparison correctly, write the results to the wrong addresses, and still leave `x4 = 0`.

The testbench closes this gap. It examines the data memory *directly*, without using any branch the firmware used. It checks six words against values calculated outside the design: words 0, 1 and 2, which stage 2 wrote with a store-word, a store-half and a store-byte; and words 4, 5 and 6, which the copy loop of stage 9 wrote. It also confirms that register `x0` is still zero after the complete run.

Words 4 to 6 prove one more thing. The copy loop reads its source data from 0x00400400, which is an address in the program memory. Correct values in those words therefore show that a load from the program memory works from end to end.

7 Video, Deliverables and Reproduction

Project demonstration video (Submission Guide §7)

<https://youtu.be/TODO-REPLACE-WITH-YOUR-VIDEO-ID>

Covering project architecture, processor operation, system simulation, firmware execution, OpenLane results and the final chip layout.

Public repository

<https://github.com/Niccc8/rvbl2-riscv-cpu-core>

The complete design, both testbench suites, the equivalence proof and every figure in this report at full resolution.

Table 10: Submission Guide §9 expected files, and where each is in the submission.

Required	Location
Report	This document
GDSII layout	The complete OpenLane run directory, under its <code>results</code> folder
Gate-level netlist	The same run directory. Both the powered and the non-powered netlist are included
Synthesis configuration	The twelve-key <code>config.json</code> that was submitted, together with the platform's own record of what it received
All code and testbenches	16 reference design files, 16 reference testbenches, and the ChipInventor set: 18 block sources, the platform testbench, the firmware and the build scripts
Video link	Above, and on the title page

Reproducing every claim

Both suites are self-checking and exit non-zero on any failure. Icarus Verilog and Python 3 are the only requirements.

Run `run_all.sh` for the reference regression: 15 testbenches and 7,384 checks. Run `run_ci.sh` for the ChipInventor chain: 12 gates, 15,281 checks, and the competition firmware verdict. Both scripts print a pass or fail line for every gate and return a non-zero exit code if any gate fails.

To repeat the physical run, enter the twelve configuration keys into the ChipInventor synthesis page. The same result can be produced locally with the pinned toolchain: OpenLane build 3876562d with the sky130A process kit bdc9412b. Two setup scripts in the submission do this.

Conclusion

RVBL-2 implements all 47 instructions that the competition specifies. It uses the multicycle arrangement that the Block Guide requires, and it was built on the ChipInventor platform. The result is a signed-off Sky130A layout of 0.404 mm^2 at 52.28 % density, with zero design-rule violations, a unique LVS match, and 2.25 ns of setup margin at the slow corner. The organisers' own validation firmware completes all nine of its stages and returns its PASS verdict.

The most important property is not any single number. It is the chain that connects them (Figure 2). The netlist that OpenLane built is provably the same netlist that the testbenches verified. Two checks prove this: a cycle-by-cycle comparison over 410,400 points, and a structural comparison of the platform's own export that does not depend on instance names. Neither is an assertion.

The design has four real limits. This report states each one with its measurement.

1. The place-and-route netlist contains 8 data words, not 2,048. The organisers instructed this reduction. Simulation shows that 8 words is the smallest size that still passes validation.
2. 86 antenna pins remain. The standard remedy was measured. It costs 19.5 % of the die area and removes none of them.
3. The tool reports 30.3 MHz. That figure repeats the constraint we set. The frequency the run actually demonstrates is about 32.5 MHz, and repeated runs vary by 1.6 ns.
4. No timing-annotated simulation was run after layout.

Result against each Submission Guide topic

§	Topic	Measured result
1	Project Overview	47 instructions in a 6-state multicycle core; 17 block definitions placed as 20 instances; 3 to 5 cycles per instruction
2	Supported Instructions	47 of 47 implemented, all ten categories at 100 %, each with a directed test that passes
3	Datapath	One ALU, one memory port and five multiplexers, shared over time by the state machine; every net in Figure 5 is real
4	Modules and Testbenches	15,281 checks in the platform suite and 8,285 in the reference suite, 0 failures; 12 automatic gates all pass
5	OpenLane Results	0.404 mm^2 at 52.28 % density; 0 DRC from two independent checkers; LVS matches uniquely; +2.25 ns setup margin at the slow corner
6	Firmware Demo	All nine validation stages complete and the firmware returns <code>x4 = 0x00000000</code> after 991 cycles

Why the two test suites are independent

The reference suite and the platform suite were written against the same specification but not against each other. The reference suite tests sixteen separate design files with expected values calculated in Python. The platform suite tests the single combined file that ChipInventor actually runs, with expected values calculated by reference functions inside the testbench. The two use different sources for their expected values, so a fault would have to appear in both, in the same way, to pass unnoticed.

A third check is stronger still, because it needs no expected values at all. The mirror equivalence test runs the canvas design and the reference design side by side for 410,400 cycles and compares the complete architectural state at every cycle. It reports zero mismatches. Whatever the two designs do, they do the same thing.